CONTENTS

INTRODUCTION

The project aims to solve the problem of finding optimal flight routes between airports by modeling the system as a graph and applying various search and sorting algorithms. The flight and airport data is loaded once at the start and is treated as static during the application's runtime.

ALGORITHM CHOICES AND REASONING

ROUTE CALCULATION ALGORITHMS

DIJKSTRA'S ALGORITHM

We chose Dijkstra because it is ideal for finding the cheapest, fastest routes with the fewest stopovers in a weighted, non-negative graph.

- Cheapest Route: By setting the edge weight to the price of each flight, Dijkstra's algorithm guarantees finding the path with the minimum cumulative cost.
- Fastest Route: By using duration (plus a fixed stopover time) as the edge weight, the algorithm identifies the route with the minimum total travel time.
- Fewest Stopovers: By assigning a uniform weight of 1 to every flight, the shortest path corresponds to the route with the minimum number of flights, and therefore the fewest stopovers.

An efficient implementation using a PriorityQueue was chosen to ensure optimal performance, reducing the time complexity from $O(V^2)$ to $O(E + V \log V)$, where V is the number of vertices (airports) and E is the number of edges (flights).

MODIFIED DEPTH-FIRST SEARCH

For the slowest route we implemented a modified Depth-First Search (DFS). The goal is to find the longest path in the graph. The search is constrained by a depth limit of stopovers (flights). This heuristic prevents prohibitively long search times and avoids cycles while still providing a "slowest" route.

SORTING ALGORITHMS

The application implements two sorting algorithms to demonstrate different trade-offs in sorting a list of Route objects. Both sorting algorithms were implemented in the class RoutSort.

MERGE SORT

Merge Sort was chosen as the stable sorting algorithm. Its key advantages are:

- Stability: It preserves the relative order of elements with equal keys. This is important when sorting by multiple criteria (e.g., sorting by price, then duration), as the initial order is maintained for routes with the same price.
- Guaranteed Performance: It has a predictable time complexity of O(n log n) in all cases (worst, average, and best), making it reliable for any dataset

QUICK SORT

As our second sorting algorithm we used Quick Sort, because of its speed and efficiency.

- In-Place Sorting: It sorts directly in the given array, requiring only O(log n) space for the recursive calls, which is more memory efficient than Merge Sort.
- Fast Average Performance: Its average case time complexity is O(n log n), making it one of the fastest sorting algorithms for our purpose.

TIME AND SPACE COMPLEXITY ANALYSIS

The following table summarizes the complexity analysis for the key algorithms used in the application.

Algorithm	Time Complexity (Average)	Time Complexity (Worst)	Space Complexity
Dijkstra	O(E + V log V)	O(E + V log V)	O(V + E)
Merge Sort	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	$O(n^2)$	O(log n)

Where V is the number of vertices (airports) and E is the number of edges (flights).

USED CLASSES

The project is organized into several packages, each with a distinct responsibility, promoting modularity and maintainability.

Class	Package	Role
Airport, Flight, Route	model	Define the core data structures. Airport is a node, Flight is an edge, and Route is a collection of flights.
FlightGraph	graph	Implements the graph data structure using an adjacency list, representing the network of airports and flights.
DataLoader	util	Handles all data ingestion and persistence, reading from and writing to CSV files.
RouteFinder	algorithm	Contains the core route-finding logic, including Dijkstra's algorithm and the modified DFS for slowest route.
RouteComparators	algorithm	Provides Comparator implementations for sorting routes by price, duration, stopovers, and a combined criteria.
RouteSorter	algorithm	Implements the Merge Sort (stable) and Quick Sort (unstable) algorithms for sorting lists of Route objects.
FlightSearch	search	Implements linear search functionality to find flights based on various criteria like origin, airline, or flight number.
FlightPlannerApp	menu	The main application class, responsible for the interactive console menu and orchestrating the overall user experience.

DATA STRUCTURE

The primary data structure for representing the flight network is an Adjacency List, implemented as a *Map<Airport, List<Flight>>*. This choice was made for several reasons:

- Efficiency for Sparse Graphs: Flight networks are typically sparse (the number of flights from an airport is much smaller than the total number of airports). Adjacency lists are more space-efficient than adjacency matrices for sparse graphs.
- Fast Neighbor Traversal: It allows for efficient retrieval of all outgoing flights from a given airport, which is a common operation in the route-finding algorithms.

For quick lookups, several Map structures are used:

- *Map<String, Airport>*: Allows for O(1) average-time access to by their IATA code. Airport objects by their IATA code.
- *Map<Integer, Flight>*: Provides O(1) average-time access to Flight objects by their unique ID.

STOPOVER HANDLING

A minimum stopover time of minutes is automatically added between connecting flights when calculating the total duration of a route. This is a realistic assumption that accounts for the time required to deplane, transfer, and board the next flight.